AegisFlow

Real-Time Transaction Risk & AML Investigation Platform

Milestone 2 — Complete Implementation Guide

Milestone 2 builds the feature store every AegisFlow model depends on: one definitions module compiled two ways - a Lua script that updates Redis atomically on every transaction, and a point-in-time batch builder for training - proven identical by an exact-equality parity test, plus the entity graph and an hourly job that turns it into graph features without leaking labels into training.

Item	Value
Milestone	M2 - feature definitions module, offline builder, online Redis path + parity tests, entity graph + graph features
Starts from	Tag v0.1.0-M1 (compose, schema/migrations, synthetic generator, IEEE-CIS ETL)
Ends with	Tag v0.2.0-M2; ingestion on :8001, graph job publishing hourly
Single source	ml/src/aegis_ml/features/definitions.py - 19 features
Online store	Redis: feat:acct:*, vel:acct:*:{1m,1h,24h,7d}, dist:*, share:*, gfeat:acct:*
Key endpoint	POST /api/v1/transactions -> {id, decision: null, duplicate}
Parity proof	tests/m2/unit/test_m2_parity.py - exact float equality, 5,000 transactions
Graph	entity_edges (unique key) -> NetworkX -> gfeat:acct:* + graph_feature_snapshots
Branch	feature/m2-feature-store

What M1 handed you

PostgreSQL with the spec section 12 tables (accounts, devices, ips, merchants, entity_edges, transactions, ground_truth_labels, users, audit_log, sanctions_list), the synthetic generator with fraud rings and laundering typologies, and the IEEE-CIS DVC stage. M2 adds the feature store; the decision field stays null until M4's scoring service.

M2 at a glance

Twelve steps, one commit each. The parity test in 2.6 is the milestone's heart: the Lua path and the offline builder must agree on every feature of every transaction, bit for bit, or the model M3 trains will see different numbers in production than in training.

Step	What you build	Key file(s)
2.1	Branch, dependencies, settings, Makefile targets	config.py, Makefile, .env
2.2	Migration: unique edges, graph snapshots, known fraud	m2_0001_feature_store.py
2.3	Feature definitions - the single source	definitions.py
2.4	Offline point-in-time builder + hand-computed tests	offline.py, test_m2_offline.py
2.5	Online path: one atomic Lua script	online_features.py
2.6	Offline/online parity proof	support.py, test_m2_parity.py
2.7	Ingestion API and persistence	schemas.py, persistence.py, api/routes.py
2.8	API contract tests	test_m2_api.py
2.9	Hourly graph feature job	graph_features.py, test_m2_graph.py
2.10	Known-fraud seed (pre-T0) and DVC stage	seed_known_fraud.py, dvc.yaml
2.11	Database integration tests	test_m2_db.py

Step	What you build	Key file(s)
2.12	Containers, verification and the v0.2.0-M2 tag	Dockerfiles, verify_m2.py

repo tree (new in M2)

```
docker-compose.m2.yml                ingestion + graph-job overlay
db/migrations/versions/m2_0001_feature_store.py
db/seed/seed_known_fraud.py
docs/adr/0001-feature-store.md
scripts/verify_m2.py
services/ingestion/
  Dockerfile  requirements.txt
  src/ingestion/  config.py schemas.py persistence.py online_features.py deps.py main.py
                  api/routes.py
ml/Dockerfile.graph  ml/requirements-graph.txt
ml/src/aegis_ml/features/  definitions.py offline.py graph_features.py
tests/m2/  support.py conftest.py  unit/ api/ integration/
```

M1 contract - check before you start

Assumption	Check	If different
Compose services `postgres` and `redis`	`docker compose config --services`	Edit depends_on and hostnames in docker-compose.m2.yml
transactions has the section 12 columns; channel accepts card / transfer / wallet	`\d transactions` in psql	Adjust INSERT_TXN in persistence.py
transactions.status is nullable or has a default	`\d transactions`	Add status to INSERT_TXN with a value your enum accepts
entity_edges(src_type, src_id, dst_type, dst_id, edge_type, first_seen, weight)	`\d entity_edges`	Adjust UPSERT_EDGE and the migration's key
ground_truth_labels(transaction_id, is_fraud)	`\d ground_truth_labels`	Edit LABEL_SQL in seed_known_fraud.py (one query)
Alembic head	`alembic heads`	Put it in m2_0001's down_revision

Decisions you must be able to defend

- **Event time (D1).** Windows use the transaction's own `ts`, passed into Lua - never Redis `TIME`. The generator replays at 10x, so wall-clock windows would disagree with offline.
- **Window rule (D2).** Velocity windows are `(ts - w, ts]`, current transaction included, ties ordered by `(ts, id)` in both paths.
- **Idempotency (D3).** `SET seen:txn:{id} NX` inside the script; integer minor units for money.
- **Profile before update (D4)** for the z-score, Welford's method, same order offline.
- **No label leakage (D5, D7).** Known fraud only from labels before cutoff T0; graph features join the snapshot that existed at each transaction's time.
- **Unique edges (D6)**, upserted only when the transaction row is new, so retries never inflate weights. Entity upsert endpoints are deferred (D8).

2.1 Branch, dependencies, settings and Makefile targets

terminal

```
git checkout develop && git pull
git checkout -b feature/m2-feature-store v0.1.0-M1
mkdir -p services/ingestion/src/ingestion/api ml/src/aegis_ml/features db/seed scripts \
         docs/adr tests/m2/unit tests/m2/api tests/m2/integration
touch services/ingestion/src/ingestion/__init__.py services/ingestion/src/ingestion/api/__init__.py \
      ml/src/aegis_ml/__init__.py ml/src/aegis_ml/features/__init__.py tests/__init__.py \
      tests/m2/__init__.py tests/m2/unit/__init__.py tests/m2/api/__init__.py \
      tests/m2/integration/__init__.py
python -m pip install "fastapi==0.115.6" "uvicorn[standard]==0.32.1" "pydantic==2.9.2" \
  "sqlalchemy==2.0.36" "psycopg[binary]==3.2.3" "alembic==1.14.0" "redis==5.2.0" \
  "loguru==0.7.3" "prometheus-client==0.21.1" "numpy==1.26.4" "pandas==2.2.3" \
  "pyarrow==18.1.0" "networkx==3.4.2" "scipy==1.14.1" "pytest==8.3.4" "httpx==0.27.2" \
  "fakeredis[lua]==2.26.1"
```

SciPy is required: NetworkX 3.x computes PageRank with it. `fakeredis[lua]` lets the Lua script run in unit tests.

services/ingestion/src/ingestion/config.py

```
"""Ingestion settings from environment variables (dev-only defaults; real values in .env)."""
import os

DATABASE_URL = os.getenv("DATABASE_URL",
                         "postgresql+psycopg://aegisflow:aegisflow@localhost:5432/aegisflow")
REDIS_URL = os.getenv("REDIS_URL", "redis://localhost:6379/0")
SERVICE_KEY = os.getenv("SERVICE_KEY", "dev-service-key")
```

env.m2.example (append to .env.example and to your real .env)

```
# Append to .env.example and your real .env (never commit real values).
SERVICE_KEY=change-me-service-key
# Host-side make targets:
DATABASE_URL=postgresql+psycopg://<POSTGRES_USER>:<POSTGRES_PASSWORD>@localhost:5432/<POSTGRES_DB>
REDIS_URL=redis://localhost:6379/0
# Optional; default = 10th percentile of transaction timestamps
# KNOWN_FRAUD_CUTOFF=2026-09-08T00:00:00Z
```

Makefile (append below the M1 block)

```
# ---- M2 targets: append below the M1 block. RECIPEPREFIX applies only to rules
# written after it, so M1's tab-indented rules keep working. ----
.RECIPEPREFIX = >
PY ?= python
M2_PATH := PYTHONPATH=services/ingestion/src:ml/src:.
M2_COMPOSE := docker compose -f docker-compose.yml -f docker-compose.m2.yml

up-m2:
> $(M2_COMPOSE) up -d --build ingestion graph-job
migrate:
> $(M2_PATH) alembic upgrade head
seed-fraud:
> $(M2_PATH) $(PY) db/seed/seed_known_fraud.py
ingest:
> $(M2_PATH) uvicorn ingestion.main:app --reload --port 8001
graph-once:
> $(M2_PATH) $(PY) -m aegis_ml.features.graph_features --once
features-pit:
> dvc repro features_pit
parity:
> $(M2_PATH) PARITY_REDIS_URL=redis://localhost:6379/15 pytest -q tests/m2/unit/test_m2_parity.py
test-m2:
> $(M2_PATH) pytest -q tests/m2/unit tests/m2/api
test-m2-int:
> $(M2_PATH) pytest -q tests/m2/integration
verify-m2:
> $(M2_PATH) $(PY) scripts/verify_m2.py
m2: test-m2 test-m2-int verify-m2
.PHONY: up-m2 migrate seed-fraud ingest graph-once \
        features-pit parity test-m2 test-m2-int verify-m2 m2
```

Why .RECIPEPREFIX is set inside the M2 block

Copying from a PDF turns tabs into spaces. `.RECIPEPREFIX = >` applies only to rules written after it, so M1's tab-indented rules keep working and every M2 rule survives copy-paste.

terminal

```
set -a; . ./.env; set +a
make -n m2          # prints the commands; proves the Makefile parses
```

```
$ git add -A && git commit -m "[M2-AegisFlow] config: dependencies, settings and M2 make targets"
```

2.2 Migration: unique edges, graph snapshots and known fraud

Three schema changes. The unique edge index makes `ON CONFLICT` upserts possible (D6); duplicates M1 may already have written are merged first (weights summed, earliest `first_seen` kept) so the index can be built. Snapshots (D7) and known fraud (D5) get their own tables.

db/migrations/versions/m2_0001_feature_store.py

```
"""M2: feature store support - unique edge index, graph snapshots, known fraud.

Revision ID: m2_0001
Revises: <M1 head>
"""
import sqlalchemy as sa
from alembic import op

revision = "m2_0001"
down_revision = None        # REQUIRED: replace with the output of `alembic heads` (M1's head)
branch_labels = None
depends_on = None

EDGE_KEY = "src_type, src_id, dst_type, dst_id, edge_type"


def upgrade():
    # 1. Merge duplicate edges M1 may have written, so the unique index can be built.
    op.execute(f"""
        UPDATE entity_edges k SET weight = d.total, first_seen = d.first
        FROM (SELECT min(id) AS keep_id, sum(weight) AS total, min(first_seen) AS first
              FROM entity_edges GROUP BY {EDGE_KEY} HAVING count(*) > 1) d
        WHERE k.id = d.keep_id""")
    op.execute(f"""
        DELETE FROM entity_edges a USING entity_edges b
        WHERE a.id > b.id AND (a.src_type, a.src_id, a.dst_type, a.dst_id, a.edge_type)
                            = (b.src_type, b.src_id, b.dst_type, b.dst_id, b.edge_type)""")
    op.create_index("uq_entity_edges_key", "entity_edges",
                    ["src_type", "src_id", "dst_type", "dst_id", "edge_type"], unique=True)
    # 2. Point-in-time graph features (Decision D7).
    op.create_table(
        "graph_feature_snapshots",
        sa.Column("entity_id", sa.Uuid(), nullable=False),
        sa.Column("computed_at", sa.DateTime(timezone=True), nullable=False),
        sa.Column("component_size", sa.Integer(), nullable=False),
        sa.Column("dist_to_fraud", sa.Integer(), nullable=False),
        sa.Column("risk_rank", sa.Float(), nullable=False),
        sa.PrimaryKeyConstraint("entity_id", "computed_at"),
    )
    op.create_index("ix_graph_snapshots_time", "graph_feature_snapshots", ["computed_at"])
    # 3. Known fraud used to seed graph features (Decision D5).
    op.create_table(
        "known_fraud_entities",
        sa.Column("entity_type", sa.String(16), nullable=False),
        sa.Column("entity_id", sa.Uuid(), nullable=False),
        sa.Column("confirmed_at", sa.DateTime(timezone=True), nullable=False),
        sa.Column("source", sa.String(32), nullable=False),
        sa.PrimaryKeyConstraint("entity_type", "entity_id"),
    )


def downgrade():
    op.drop_table("known_fraud_entities")
    op.drop_index("ix_graph_snapshots_time", table_name="graph_feature_snapshots")
    op.drop_table("graph_feature_snapshots")
    op.drop_index("uq_entity_edges_key", table_name="entity_edges")
```

db/migrations/env.py (only if M1 has none)

```
"""Minimal Alembic env - use only if M1 did not create one. URL from DATABASE_URL."""
import os
from logging.config import fileConfig

from alembic import context
from sqlalchemy import engine_from_config, pool

config = context.config
if config.config_file_name:
    fileConfig(config.config_file_name)
config.set_main_option("sqlalchemy.url", os.environ["DATABASE_URL"])


def run_online():
    engine = engine_from_config(config.get_section(config.config_ini_section),
                                prefix="sqlalchemy.", poolclass=pool.NullPool)
    with engine.connect() as conn:
        context.configure(connection=conn, target_metadata=None)
        with context.begin_transaction():
            context.run_migrations()


run_online()
```

terminal

```
alembic heads                    # put this id into down_revision first
make migrate
# Running upgrade <M1 head> -> m2_0001 ...
docker compose exec postgres psql -U $POSTGRES_USER -d $POSTGRES_DB -c "\d entity_edges"
# ... "uq_entity_edges_key" UNIQUE, btree (src_type, src_id, dst_type, dst_id, edge_type)
```

```
$ git add -A && git commit -m "[M2-AegisFlow] feat: unique edge key, graph snapshots and known-fraud tables"
```

2.3 Feature definitions - the single source of truth

Every feature is declared once: name, entity, kind, window, include-current rule, default and which path computes it. Both paths read windows, defaults and output order from here, and `FEATURE_SET_VERSION` hashes the whole set so M3's model card can name it exactly.

ml/src/aegis_ml/features/definitions.py

```
"""SINGLE SOURCE OF TRUTH for every AegisFlow feature (FR-3, spec section 11).

The online path (services/ingestion/.../online_features.py, a Lua script) and the
offline path (offline.py, batch over a table) both read the windows, defaults and
output order from here. Parity rules (ADR 0001):
  D1 event time: windows use the transaction's own ts, never server time.
  D2 velocity windows are (ts - w, ts], current transaction INCLUDED; ties are
     ordered by (ts, id) in both paths.
  D4 amount_zscore uses the account profile BEFORE the current amount is added."""
import hashlib
import json
from dataclasses import asdict, dataclass
from datetime import datetime, timedelta, timezone
from decimal import Decimal

WINDOWS = {"1m": 60, "1h": 3_600, "24h": 86_400, "7d": 604_800}   # ordered
DISTINCT_WINDOW_S = 86_400          # distinct devices / IPs per account
SHARE_WINDOW_S = 604_800            # distinct accounts per device / IP
KEY_TTL_S = 8 * 86_400              # every Redis key outlives the longest window
GRAPH_TTL_S = 2 * 3_600             # gfeat:* outlives one missed hourly run


@dataclass(frozen=True)
class Feature:
    name: str
    entity: str          # account | device | ip
    kind: str            # count | sum | distinct | zscore | recency | calendar | graph
    window_s: int | None
    include_current: bool
    default: float
    source: str          # lua | calendar | graph
```

ml/src/aegis_ml/features/definitions.py

```
def _velocity():
    out = []
    for label, w in WINDOWS.items():
        out.append(Feature(f"txn_count_{label}", "account", "count", w, True, 0.0, "lua"))
        out.append(Feature(f"amount_sum_minor_{label}", "account", "sum", w, True, 0.0, "lua"))
    return out


FEATURES: list[Feature] = _velocity() + [
    Feature("distinct_devices_24h", "account", "distinct", DISTINCT_WINDOW_S, True, 0.0, "lua"),
    Feature("distinct_ips_24h", "account", "distinct", DISTINCT_WINDOW_S, True, 0.0, "lua"),
    Feature("device_accounts_7d", "device", "distinct", SHARE_WINDOW_S, True, 0.0, "lua"),
    Feature("ip_accounts_7d", "ip", "distinct", SHARE_WINDOW_S, True, 0.0, "lua"),
    Feature("amount_zscore", "account", "zscore", None, False, 0.0, "lua"),
    Feature("seconds_since_prev", "account", "recency", None, False, -1.0, "lua"),
    Feature("hour_of_day", "account", "calendar", None, True, 0.0, "calendar"),
    Feature("day_of_week", "account", "calendar", None, True, 0.0, "calendar"),
    Feature("graph_component_size", "account", "graph", None, False, 1.0, "graph"),
    Feature("graph_dist_to_fraud", "account", "graph", None, False, -1.0, "graph"),
    Feature("graph_risk_rank", "account", "graph", None, False, 0.0, "graph"),
]
FEATURE_LIST = [f.name for f in FEATURES]
BY_NAME = {f.name: f for f in FEATURES}
LUA_OUTPUTS = [f.name for f in FEATURES if f.source == "lua"]      # order the script returns
GRAPH_FIELDS = {"graph_component_size": "component_size",
                "graph_dist_to_fraud": "dist_to_fraud",
                "graph_risk_rank": "risk_rank"}
FEATURE_SET_VERSION = hashlib.sha256(json.dumps(
    [asdict(f) for f in FEATURES] + [WINDOWS, DISTINCT_WINDOW_S, SHARE_WINDOW_S],
    sort_keys=True).encode()).hexdigest()[:12]

_EPOCH = datetime(1970, 1, 1, tzinfo=timezone.utc)


def epoch_seconds(ts: datetime) -> float:
    """The ONE conversion from timestamp to window clock, used by both paths."""
    return ((ts - _EPOCH) // timedelta(microseconds=1)) / 1_000_000


def calendar(ts: datetime) -> dict:
    u = ts.astimezone(timezone.utc)
    return {"hour_of_day": float(u.hour), "day_of_week": float(u.weekday())}


def amount_minor(amount) -> int:
    """Decimal or 2-dp float -> integer minor units (paise/cents)."""
    if isinstance(amount, Decimal):
        return int((amount * 100).to_integral_value())
    return int(round(float(amount) * 100))
```

```
$ git add -A && git commit -m "[M2-AegisFlow] feat: single-source feature definitions"
```

2.4 Offline point-in-time builder

Explicit two-pointer loops so every boundary is visible: a row only sees transactions at or before itself in `(ts, id)` order. The graph join uses `merge_asof` backward on `computed_at`, so a transaction only sees a snapshot that already existed.

ml/src/aegis_ml/features/offline.py

```
"""Offline, point-in-time feature builder (FR-3). Same definitions, same rules as the
Lua path, written as explicit two-pointer loops so every boundary is visible:
a row only ever sees transactions at or before itself in (ts, id) order.

    python -m aegis_ml.features.offline --out data/features/features_pit.parquet"""
import argparse
import math
import os
import pathlib
from collections import Counter

import numpy as np
import pandas as pd

from . import definitions as D

COLUMNS = ["id", "ts", "account_id", "amount_minor", "device_id", "ip_id"]


def prepare(txns: pd.DataFrame) -> pd.DataFrame:
    """Normalise to COLUMNS and sort by (ts, id) - the arrival order the online path sees."""
    f = txns.copy()
    if "amount_minor" not in f:
        f["amount_minor"] = [D.amount_minor(a) for a in f["amount"]]
    for c in ("id", "account_id", "device_id", "ip_id"):
        f[c] = f[c].map(lambda v: None if v is None or (isinstance(v, float) and math.isnan(v))
                        else str(v))
    f["ts"] = pd.to_datetime(f["ts"], utc=True)
    f["t"] = [D.epoch_seconds(x.to_pydatetime()) for x in f["ts"]]
    return f.sort_values(["t", "id"], kind="mergesort").reset_index(drop=True)


def _windowed_counts(times, amounts, w):
    """count and sum over (t_i - w, t_i] for rows of ONE account, in order."""
    n = len(times)
    cnt, sm = np.zeros(n), np.zeros(n)
    prefix = np.concatenate([[0], np.cumsum(amounts)])
    left = 0
    for i in range(n):
        cut = times[i] - w
        while times[left] <= cut:
            left += 1
        cnt[i] = i - left + 1
        sm[i] = prefix[i + 1] - prefix[left]
    return cnt, sm


def _windowed_distinct(times, keys, w):
    """distinct non-null keys over (t_i - w, t_i], rows in order."""
    n = len(times)
    out = np.zeros(n)
    live, left = Counter(), 0
    for i in range(n):
        cut = times[i] - w
        while times[left] <= cut:
            if keys[left] is not None:
                live[keys[left]] -= 1
                if live[keys[left]] == 0:
                    del live[keys[left]]
            left += 1
        if keys[i] is not None:
            live[keys[i]] += 1
        out[i] = len(live)
    return out


def _profile(amounts_minor, times):
    """amount_zscore (profile BEFORE the row) and seconds_since_prev, Welford order
    identical to the Lua script."""
    z, since = np.zeros(len(times)), np.full(len(times), -1.0)
    n, mean, m2, last = 0, 0.0, 0.0, None
    for i, (a, t) in enumerate(zip(amounts_minor, times)):
        x = a / 100
        if n >= 2:
            sd = math.sqrt(m2 / (n - 1))
            if sd > 0:
                z[i] = (x - mean) / sd
        if last is not None:
            since[i] = t - last
        n += 1
        delta = x - mean
```

ml/src/aegis_ml/features/offline.py

```
        mean = mean + delta / n
        m2 = m2 + delta * (x - mean)
        last = t
    return z, since


def build(txns: pd.DataFrame, snapshots: pd.DataFrame | None = None) -> pd.DataFrame:
    f = prepare(txns)
    out = pd.DataFrame({"id": f["id"], "ts": f["ts"], "account_id": f["account_id"]})
    for name in D.FEATURE_LIST:
        out[name] = float(D.BY_NAME[name].default)
    for _, g in f.groupby("account_id", sort=False):
        idx = g.index.to_numpy()
        t, amt = g["t"].to_numpy(), g["amount_minor"].to_numpy(dtype=np.int64)
        for label, w in D.WINDOWS.items():
            c, s = _windowed_counts(t, amt, w)
            out.loc[idx, f"txn_count_{label}"] = c
            out.loc[idx, f"amount_sum_minor_{label}"] = s
        out.loc[idx, "distinct_devices_24h"] = _windowed_distinct(
            t, g["device_id"].tolist(), D.DISTINCT_WINDOW_S)
        out.loc[idx, "distinct_ips_24h"] = _windowed_distinct(
            t, g["ip_id"].tolist(), D.DISTINCT_WINDOW_S)
        z, since = _profile(amt, t)
        out.loc[idx, "amount_zscore"] = z
        out.loc[idx, "seconds_since_prev"] = since
    for col, feat in (("device_id", "device_accounts_7d"), ("ip_id", "ip_accounts_7d")):
        for _, g in f[f[col].notna()].groupby(col, sort=False):
            out.loc[g.index, feat] = _windowed_distinct(
                g["t"].to_numpy(), g["account_id"].tolist(), D.SHARE_WINDOW_S)
    cal = pd.DataFrame([D.calendar(x.to_pydatetime()) for x in f["ts"]], index=f.index)
    out[["hour_of_day", "day_of_week"]] = cal[["hour_of_day", "day_of_week"]]
    if snapshots is not None and len(snapshots):
        out = attach_graph(out, snapshots)
    out.attrs["feature_set_version"] = D.FEATURE_SET_VERSION
    return out[["id", "ts", "account_id"] + D.FEATURE_LIST]


def attach_graph(feat: pd.DataFrame, snapshots: pd.DataFrame) -> pd.DataFrame:
    """Latest graph snapshot with computed_at <= ts per account (Decision D7)."""
    s = snapshots.rename(columns={"entity_id": "account_id"}).copy()
    s["account_id"] = s["account_id"].astype(str)
    s["computed_at"] = pd.to_datetime(s["computed_at"], utc=True)
    s = s.sort_values("computed_at")
    left = feat.drop(columns=list(D.GRAPH_FIELDS)).reset_index().sort_values("ts")
    merged = pd.merge_asof(left, s, left_on="ts", right_on="computed_at",
                           by="account_id", direction="backward")
    for name, col in D.GRAPH_FIELDS.items():
        merged[name] = merged[col].astype(float).fillna(D.BY_NAME[name].default)
    return merged.sort_values("index").set_index("index").rename_axis(None)[feat.columns]


def main():
    import sqlalchemy as sa
    ap = argparse.ArgumentParser()
    ap.add_argument("--out", required=True)
    a = ap.parse_args()
    eng = sa.create_engine(os.environ["DATABASE_URL"])
    txns = pd.read_sql("SELECT id, ts, account_id, amount, device_id, ip_id FROM transactions",
                       eng)
    snaps = pd.read_sql("SELECT entity_id, computed_at, component_size, dist_to_fraud, "
                        "risk_rank FROM graph_feature_snapshots", eng)
    out = build(txns, snaps)
    pathlib.Path(a.out).parent.mkdir(parents=True, exist_ok=True)
    out.to_parquet(a.out, index=False)
    print(f"{len(out)} rows x {len(D.FEATURE_LIST)} features "
          f"(feature set {D.FEATURE_SET_VERSION}) -> {a.out}")


if __name__ == "__main__":
    main()
```

tests/m2/conftest.py

```
"""M2 tests: shared builders live in tests/m2/support.py."""
```

tests/m2/unit/test_m2_offline.py

```
"""Hand-computed fixture: window edges, ties, include-current and the z-score order."""
from datetime import datetime, timedelta, timezone

import pandas as pd
import pytest

from aegis_ml.features import definitions as D
from aegis_ml.features.offline import attach_graph, build

T = datetime(2026, 9, 1, 12, 0, tzinfo=timezone.utc)
A, B, DEV = "a", "b", "d1"


def txn(i, sec, acct, minor, dev=DEV):
    return {"id": f"t{i}", "ts": T + timedelta(seconds=sec), "account_id": acct,
            "amount_minor": minor, "device_id": dev, "ip_id": None}


def frame():
    return pd.DataFrame([txn(1, 0, A, 1000), txn(2, 30, A, 2000), txn(3, 30, A, 3000),
                         txn(4, 60, A, 4000), txn(5, 61, B, 500), txn(6, 120, A, 1000, None)])


def test_velocity_edges_and_ties():
    f = build(frame()).set_index("id")
    assert f.loc["t3", "txn_count_1m"] == 3          # t1, t2 and itself (tie with t2)
    assert f.loc["t2", "txn_count_1m"] == 2          # t3 comes AFTER t2 in (ts, id) order
    assert f.loc["t4", "txn_count_1m"] == 3          # t1 sits exactly on the edge: excluded
    assert f.loc["t4", "amount_sum_minor_1m"] == 9000
    assert f.loc["t6", "txn_count_1m"] == 1          # 120 - 60 = 60 > t4: only itself


def test_sharing_and_distinct():
    f = build(frame()).set_index("id")
    assert f.loc["t5", "device_accounts_7d"] == 2    # a and b on d1
    assert f.loc["t6", "device_accounts_7d"] == 0    # no device
    assert f.loc["t6", "distinct_devices_24h"] == 1  # d1 still inside 24h


def test_zscore_uses_profile_before_current():
    f = build(frame()).set_index("id")
    assert f.loc["t1", "amount_zscore"] == 0.0 and f.loc["t2", "amount_zscore"] == 0.0
    mean, sd = 15.0, (50.0) ** 0.5                # profile after t1, t2 = 10, 20
    assert f.loc["t3", "amount_zscore"] == pytest.approx((30 - mean) / sd)
    assert f.loc["t1", "seconds_since_prev"] == -1.0
    assert f.loc["t4", "seconds_since_prev"] == 30.0


def test_graph_join_is_point_in_time():
    snaps = pd.DataFrame([
        {"entity_id": A, "computed_at": T + timedelta(seconds=45), "component_size": 4,
         "dist_to_fraud": 2, "risk_rank": 0.1}])
    f = attach_graph(build(frame()), snaps).set_index("id")
    assert f.loc["t2", "graph_component_size"] == 1.0          # before the snapshot: default
    assert f.loc["t4", "graph_component_size"] == 4.0          # after it: snapshot value
    assert f.loc["t5", "graph_dist_to_fraud"] == -1.0          # account b: no snapshot
    assert list(f.columns[2:]) == D.FEATURE_LIST
```

terminal

```
PYTHONPATH=services/ingestion/src:ml/src:. pytest -q tests/m2/unit/test_m2_offline.py
# 4 passed
```

```
$ git add -A && git commit -m "[M2-AegisFlow] feat: offline point-in-time builder with hand-computed tests"
```

2.5 Online path: one atomic Lua script

Redis runs a script without interleaving, so each transaction's update is atomic. The script trims each window to `(ts - w, ts]`, keeps running sums in the account hash (subtracting what expires), counts distinct devices/IPs and shared accounts, and computes the z-score from the profile before adding the new amount. Floats leave Redis as `%.17g` strings, which round-trip bit-exactly.

services/ingestion/src/ingestion/online_features.py

```
"""Online feature path: ONE Lua script per transaction, compiled from definitions.py.
Atomic (Redis runs a script without interleaving), idempotent (seen:txn:{id} NX guard,
Decision D3) and on event time (ts passed in, Decision D1).

Keys (all expire after KEY_TTL_S):
  seen:txn:{id}              idempotency guard
  feat:acct:{id}             hash: n, mean, m2 (Welford), last_ts, sum1..sum4
  vel:acct:{id}:{1m|1h|24h|7d}   zset member "{txn_id}|{amount_minor}" score ts
  dist:acct:{id}:dev|ip      zset of devices / IPs seen by the account (score = last seen)
  share:dev:{id} / share:ip:{id}  zset of accounts seen on the device / IP
  gfeat:acct:{id}            written by the hourly graph job, read here"""
from aegis_ml.features import definitions as D

SCRIPT = r"""
local function fmt(x) return string.format('%.17g', x) end
local ts  = tonumber(ARGV[2])
local amt = tonumber(ARGV[3])
local ttl = tonumber(ARGV[9])
if not redis.call('SET', KEYS[1], '1', 'NX', 'EX', ttl) then
  return {'dup'}
end
local out = {}
local member = ARGV[1] .. '|' .. ARGV[3]

-- 1. velocity windows (ts - w, ts], running sums kept in the account hash
for i = 1, 4 do
  local k = KEYS[2 + i]
  local cut = ts - tonumber(ARGV[9 + i])
  local field = 'sum' .. i
  local s = tonumber(redis.call('HGET', KEYS[2], field)) or 0
  local expired = redis.call('ZRANGEBYSCORE', k, '-inf', fmt(cut))
  for _, m in ipairs(expired) do
    s = s - tonumber(string.match(m, '|(%-?%d+)$'))
  end
  if #expired > 0 then redis.call('ZREMRANGEBYSCORE', k, '-inf', fmt(cut)) end
  redis.call('ZADD', k, fmt(ts), member)
  s = s + amt
  redis.call('HSET', KEYS[2], field, fmt(s))
  redis.call('EXPIRE', k, ttl)
  out[#out + 1] = fmt(redis.call('ZCARD', k))
  out[#out + 1] = fmt(s)
end

-- 2. distinct counts: trim to the window, add the current entity, count
local function distinct(k, w, m, has)
  redis.call('ZREMRANGEBYSCORE', k, '-inf', fmt(ts - w))
  if has == '1' then redis.call('ZADD', k, fmt(ts), m) end
  redis.call('EXPIRE', k, ttl)
  return redis.call('ZCARD', k)
end
local dw, sw = tonumber(ARGV[14]), tonumber(ARGV[15])
out[#out + 1] = fmt(distinct(KEYS[7], dw, ARGV[7], ARGV[4]))
out[#out + 1] = fmt(distinct(KEYS[8], dw, ARGV[8], ARGV[5]))
if ARGV[4] == '1' then out[#out + 1] = fmt(distinct(KEYS[9], sw, ARGV[6], '1'))
else out[#out + 1] = '0' end
if ARGV[5] == '1' then out[#out + 1] = fmt(distinct(KEYS[10], sw, ARGV[6], '1'))
else out[#out + 1] = '0' end

-- 3. z-score from the profile BEFORE this amount (D4), then Welford update
local h = redis.call('HMGET', KEYS[2], 'n', 'mean', 'm2', 'last_ts')
local n = tonumber(h[1]) or 0
local mean = tonumber(h[2]) or 0
local m2 = tonumber(h[3]) or 0
local x = amt / 100
local z = 0
if n >= 2 then
  local sd = math.sqrt(m2 / (n - 1))
  if sd > 0 then z = (x - mean) / sd end
end
local since = -1
if h[4] then since = ts - tonumber(h[4]) end
n = n + 1
```

services/ingestion/src/ingestion/online_features.py

```
local delta = x - mean
mean = mean + delta / n
m2 = m2 + delta * (x - mean)
redis.call('HSET', KEYS[2], 'n', fmt(n), 'mean', fmt(mean), 'm2', fmt(m2), 'last_ts', fmt(ts))
redis.call('EXPIRE', KEYS[2], ttl)
out[#out + 1] = fmt(z)
out[#out + 1] = fmt(since)
return out
"""


def keys_for(t: dict) -> list[str]:
    a = t["account_id"]
    return ([f"seen:txn:{t['id']}", f"feat:acct:{a}"]
            + [f"vel:acct:{a}:{label}" for label in D.WINDOWS]
            + [f"dist:acct:{a}:dev", f"dist:acct:{a}:ip",
               f"share:dev:{t['device_id'] or '-'}", f"share:ip:{t['ip_id'] or '-'}"])


def args_for(t: dict) -> list:
    return ([str(t["id"]), repr(D.epoch_seconds(t["ts"])), str(t["amount_minor"]),
             "1" if t["device_id"] else "0", "1" if t["ip_id"] else "0",
             str(t["account_id"]), str(t["device_id"] or ""), str(t["ip_id"] or ""),
             str(D.KEY_TTL_S)]
            + [str(w) for w in D.WINDOWS.values()]
            + [str(D.DISTINCT_WINDOW_S), str(D.SHARE_WINDOW_S)])


def read_graph(r, account_id) -> dict:
    vals = r.hmget(f"gfeat:acct:{account_id}", *D.GRAPH_FIELDS.values())
    return {name: (float(v) if v is not None else D.BY_NAME[name].default)
            for name, v in zip(D.GRAPH_FIELDS, vals)}


def update(r, t: dict) -> dict | None:
    """t: {id, ts (aware datetime), account_id, amount_minor, device_id, ip_id}.
    Returns the full feature row in FEATURE_LIST order, or None for a duplicate id."""
    res = r.register_script(SCRIPT)(keys=keys_for(t), args=args_for(t))
    res = [x.decode() if isinstance(x, bytes) else x for x in res]
    if res == ["dup"]:
        return None
    row = dict(zip(D.LUA_OUTPUTS, (float(x) for x in res)))
    row.update(D.calendar(t["ts"]))
    row.update(read_graph(r, t["account_id"]))
    return {name: row[name] for name in D.FEATURE_LIST}
```

Two Redis Lua rules that cause silent bugs

A Lua number returned to the client is truncated to an integer, so every value is returned as a string. Every key the script touches must be passed in KEYS (here: 10 keys, with `share:dev:-` / `share:ip:-` placeholders that are never written when the device or IP is missing).

```
$ git add -A && git commit -m "[M2-AegisFlow] feat: atomic idempotent online feature script"
```

2.6 Offline/online parity proof

A deterministic stream of 5,000 transactions with ties, missing devices, shared devices and gaps that land exactly on the 1-minute and 1-hour edges goes through the Lua path in arrival order; the offline builder computes the same rows. Every row and every column must be equal - `check_exact=True`, no tolerance.

tests/m2/support.py

```
"""Shared builders. stream() makes a deterministic synthetic transaction stream with
ties, shared devices/IPs, missing devices and gaps that cross every window edge -
self-contained so the parity proof does not depend on the M1 generator's internals."""
import uuid
from datetime import datetime, timedelta, timezone

import numpy as np
import pandas as pd

T0 = datetime(2026, 9, 1, tzinfo=timezone.utc)
NS = uuid.UUID("0b6c6f2e-2d8a-4f53-9a51-3c1d2b7e8f90")


def stream(n=5000, accounts=300, devices=150, ips=100, seed=7) -> pd.DataFrame:
    rng = np.random.default_rng(seed)
    acc = [str(uuid.uuid5(NS, f"acct-{i}")) for i in range(accounts)]
    dev = [str(uuid.uuid5(NS, f"dev-{i}")) for i in range(devices)]
    ipl = [str(uuid.uuid5(NS, f"ip-{i}")) for i in range(ips)]
    gaps = rng.choice([0, 0, 1, 5, 30, 59, 60, 61, 600, 3600, 86_400], size=n)
    jitter = rng.integers(0, 1_000_000, size=n) / 1e6 * (rng.random(n) < 0.3)
    secs = np.cumsum(gaps) + jitter          # 70% of rows sit on whole seconds
    rows = []
    for i in range(n):
        rows.append({
            "id": str(uuid.uuid5(NS, f"txn-{seed}-{i}")),
            "ts": T0 + timedelta(microseconds=int(round(secs[i] * 1e6))),
            "account_id": acc[int(rng.zipf(1.6)) % accounts],
            "amount_minor": int(rng.integers(100, 500_000)),
            "device_id": None if rng.random() < 0.15 else dev[int(rng.integers(devices))],
            "ip_id": None if rng.random() < 0.10 else ipl[int(rng.integers(ips))],
        })
    return pd.DataFrame(rows)
```

tests/m2/unit/test_m2_parity.py

```
"""THE M2 contract: for every transaction, the Lua path and the offline builder produce
IDENTICAL features - every row, every column, exact float equality. Runs on fakeredis
always, and also on a real Redis when PARITY_REDIS_URL is set (Lua 5.1 there vs 5.4 in
fakeredis). PARITY_REDIS_URL must point at a SCRATCH database (e.g. /15): the test
flushes it. Never point it at the live feature store.
If this goes red, fix the online code, never the test."""
import os

import pandas as pd
import pytest

from aegis_ml.features import definitions as D
from aegis_ml.features.offline import build, prepare
from ingestion.online_features import update
from tests.m2.support import stream


def _redis_clients():
    import fakeredis
    clients = [("fakeredis", fakeredis.FakeRedis())]
    if os.getenv("PARITY_REDIS_URL"):
        import redis
        r = redis.Redis.from_url(os.environ["PARITY_REDIS_URL"])
        r.flushdb()
        clients.append(("redis", r))
    return clients


@pytest.mark.parametrize("name, r", _redis_clients())
def test_online_equals_offline_for_every_transaction(name, r):
    txns = stream()
    offline = build(txns).set_index("id")[D.FEATURE_LIST]
    online = {}
    for t in prepare(txns).itertuples(index=False):        # (ts, id) arrival order
        row = update(r, {"id": t.id, "ts": t.ts.to_pydatetime(), "account_id": t.account_id,
                         "amount_minor": int(t.amount_minor), "device_id": t.device_id,
                         "ip_id": t.ip_id})
        online[t.id] = row
    online = pd.DataFrame.from_dict(online, orient="index")[D.FEATURE_LIST]
    pd.testing.assert_frame_equal(offline.loc[online.index].astype(float),
                                  online.astype(float), check_exact=True)
    if name == "redis":
        r.flushdb()
```

tests/m2/unit/test_m2_parity.py

```
def test_stream_really_exercises_the_edges():
    f = prepare(stream())
    assert f["t"].duplicated().any()                        # ties
    assert f["device_id"].isna().any()                      # missing devices
    gaps = f["t"].diff()
    assert (gaps == 60).any() and (gaps == 3600).any()      # gaps exactly on window edges
```

terminal

```
make parity          # fakeredis + a real Redis scratch database (db 15)
# 3 passed             (needs Redis published on localhost:6379 - ports: ["6379:6379"])
```

Do not point PARITY_REDIS_URL at the live store

The parity test flushes the database it is given. `make parity` uses `redis://localhost:6379/15`; the live feature store is database 0. Without PARITY_REDIS_URL the test runs on fakeredis only (2 passed).

When parity breaks, the online code is wrong

The offline builder produces the tables M3 trains on, so it is the specification. Breaking the window boundary on purpose (exclusive instead of inclusive cut) turns 235 of 5,000 rows red - the test is a real guard. Fix online_features.py, never the test.

```
$ git add -A && git commit -m "[M2-AegisFlow] test: exact offline/online parity over a replayed stream"
```

2.7 Ingestion API and persistence

Order inside a request: validate, commit the transaction row and its edges in one database transaction, then run the Lua script. The script runs even for a duplicate id: if an earlier attempt committed the row but died before Redis, the retry completes the features exactly once. Redis down returns 503 so the caller retries the same request.

services/ingestion/src/ingestion/schemas.py

```
"""Wire format for POST /api/v1/transactions. Money is Decimal with at most 2 decimal
places (never float), timestamps must carry a timezone."""
from datetime import datetime
from decimal import Decimal
from typing import Literal
from uuid import UUID

from pydantic import BaseModel, Field, field_validator


class TransactionIn(BaseModel):
    id: UUID
    ts: datetime
    account_id: UUID
    counterparty_id: UUID | None = None
    merchant_id: UUID | None = None
    device_id: UUID | None = None
    ip_id: UUID | None = None
    amount: Decimal = Field(gt=0, max_digits=14, decimal_places=2)
    currency: str = Field(pattern=r"^[A-Z]{3}$")
    channel: Literal["card", "transfer", "wallet"]

    @field_validator("ts")
    @classmethod
    def _tz(cls, v: datetime) -> datetime:
        if v.tzinfo is None or v.utcoffset() is None:
            raise ValueError("ts must carry a timezone")
        return v

    def feature_input(self) -> dict:
        from aegis_ml.features.definitions import amount_minor
        return {"id": str(self.id), "ts": self.ts, "account_id": str(self.account_id),
                "amount_minor": amount_minor(self.amount),
                "device_id": str(self.device_id) if self.device_id else None,
                "ip_id": str(self.ip_id) if self.ip_id else None}


class TransactionOut(BaseModel):
    id: UUID
```

services/ingestion/src/ingestion/schemas.py

```
    decision: str | None = None        # filled by the scoring service in M4
    duplicate: bool = False
```

services/ingestion/src/ingestion/persistence.py

```
"""Transaction insert + entity-edge upserts in ONE database transaction. The edge upsert
only runs when the transaction row was actually inserted, so a retried POST never
inflates edge weights. Requires the unique index from migration m2_0001 (Decision D6)."""
from sqlalchemy import text
from sqlalchemy.exc import IntegrityError

from .schemas import TransactionIn

INSERT_TXN = text("""
INSERT INTO transactions (id, ts, account_id, counterparty_id, merchant_id, device_id, ip_id,
                          amount, currency, channel)
VALUES (:id, :ts, :account_id, :counterparty_id, :merchant_id, :device_id, :ip_id,
        :amount, :currency, :channel)
ON CONFLICT (id) DO NOTHING
RETURNING id""")

UPSERT_EDGE = text("""
INSERT INTO entity_edges (src_type, src_id, dst_type, dst_id, edge_type, first_seen, weight)
VALUES (:src_type, :src_id, :dst_type, :dst_id, :edge_type, :ts, 1)
ON CONFLICT (src_type, src_id, dst_type, dst_id, edge_type)
DO UPDATE SET weight = entity_edges.weight + 1""")


class UnknownEntity(ValueError):
    """A referenced account/device/ip/merchant id does not exist (FK violation)."""


def edges_for(t: TransactionIn) -> list[dict]:
    out = []
    for kind, value, edge in (("device", t.device_id, "uses_device"),
                              ("ip", t.ip_id, "uses_ip"),
                              ("merchant", t.merchant_id, "pays_merchant")):
        if value:
            out.append({"src_type": "account", "src_id": t.account_id, "dst_type": kind,
                        "dst_id": value, "edge_type": edge, "ts": t.ts})
    if t.channel == "transfer" and t.counterparty_id:
        out.append({"src_type": "account", "src_id": t.account_id, "dst_type": "account",
                    "dst_id": t.counterparty_id, "edge_type": "transfers_to", "ts": t.ts})
    return out


class PgStore:
    def __init__(self, engine):
        self.engine = engine

    def store(self, t: TransactionIn) -> bool:
        """True if inserted now, False if the id already existed."""
        try:
            with self.engine.begin() as conn:
                inserted = conn.execute(INSERT_TXN, t.model_dump()).first() is not None
                if inserted:
                    for e in edges_for(t):
                        conn.execute(UPSERT_EDGE, e)
                return inserted
        except IntegrityError as exc:
            if getattr(exc.orig, "sqlstate", None) == "23503":       # foreign_key_violation
                raise UnknownEntity(str(exc.orig).splitlines()[0]) from None
            raise

    def ping(self) -> bool:
        with self.engine.connect() as conn:
            conn.execute(text("SELECT 1"))
        return True


class MemoryStore:
    """Test double with the same interface."""

    def __init__(self, known_accounts=None):
        self.rows, self.edges, self.known = {}, {}, known_accounts

    def store(self, t: TransactionIn) -> bool:
        if self.known is not None and str(t.account_id) not in self.known:
            raise UnknownEntity("account not found")
        if str(t.id) in self.rows:
            return False
```

services/ingestion/src/ingestion/persistence.py

```
        self.rows[str(t.id)] = t
        for e in edges_for(t):
            k = (e["src_type"], str(e["src_id"]), e["dst_type"], str(e["dst_id"]), e["edge_type"])
            self.edges[k] = self.edges.get(k, 0) + 1
        return True

    def ping(self) -> bool:
        return True
```

services/ingestion/src/ingestion/deps.py

```
"""Process singletons; tests assign the module globals directly."""
import redis
from sqlalchemy import create_engine

from . import config
from .persistence import PgStore

_store = _redis = None


def store():
    global _store
    if _store is None:
        _store = PgStore(create_engine(config.DATABASE_URL, pool_pre_ping=True))
    return _store


def redis_client():
    global _redis
    if _redis is None:
        _redis = redis.Redis.from_url(config.REDIS_URL)
    return _redis
```

services/ingestion/src/ingestion/api/routes.py

```
"""POST /api/v1/transactions: validate -> commit row + edges -> Lua feature update.
The Lua step runs even for a duplicate id: if an earlier attempt committed the row but
died before Redis, the retry completes the features exactly once (seen:txn guard).
Redis down -> 503 so the caller retries; the decision field stays null until M4."""
import secrets

import redis as redis_lib
from fastapi import APIRouter, Depends, Header, HTTPException, Response
from loguru import logger
from prometheus_client import CONTENT_TYPE_LATEST, Counter, Histogram, generate_latest

from .. import config, deps
from ..online_features import update
from ..persistence import UnknownEntity
from ..schemas import TransactionIn, TransactionOut

router = APIRouter()
TXNS = Counter("af_transactions_total", "Transactions received", ["result"])
FEATURE_SECONDS = Histogram("af_online_feature_seconds", "Lua feature update latency")


def require_service_key(x_service_key: str | None = Header(default=None)):
    if x_service_key is None or not secrets.compare_digest(x_service_key, config.SERVICE_KEY):
        raise HTTPException(401, "invalid service key")


@router.post("/api/v1/transactions", response_model=TransactionOut,
             dependencies=[Depends(require_service_key)])
def post_transaction(t: TransactionIn, response: Response):
    try:
        inserted = deps.store().store(t)
    except UnknownEntity as exc:
        TXNS.labels("unknown_entity").inc()
        raise HTTPException(422, f"unknown entity: {exc}") from None
    try:
        with FEATURE_SECONDS.time():
            update(deps.redis_client(), t.feature_input())
    except redis_lib.RedisError as exc:
        TXNS.labels("features_pending").inc()
        logger.error("online feature update failed for {}: {}", t.id, exc)
        raise HTTPException(503, "stored; online features pending - retry the same request") from None
    TXNS.labels("created" if inserted else "duplicate").inc()
    response.status_code = 201 if inserted else 200
    return TransactionOut(id=t.id, duplicate=not inserted)
```

services/ingestion/src/ingestion/api/routes.py

```
@router.get("/health")
def health():
    return {"status": "ok"}


@router.get("/ready")
def ready():
    checks = {}
    for name, probe in (("database", lambda: deps.store().ping()),
                        ("redis", lambda: deps.redis_client().ping())):
        try:
            checks[name] = bool(probe())
        except Exception:  # noqa: BLE001
            checks[name] = False
    if not all(checks.values()):
        raise HTTPException(503, detail=checks)
    return {"status": "ready", **checks}


@router.get("/metrics")
def metrics():
    return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)
```

services/ingestion/src/ingestion/main.py

```
"""uvicorn ingestion.main:app --host 0.0.0.0 --port 8001"""
import sys

from fastapi import FastAPI
from loguru import logger

from .api import routes

logger.remove()
logger.add(sys.stdout, serialize=True, level="INFO")

app = FastAPI(title="AegisFlow ingestion", version="0.2.0")
app.include_router(routes.router)
```

terminal

```
make ingest          # uvicorn on :8001
curl -s localhost:8001/ready
# {"status":"ready","database":true,"redis":true}
make generate        # M1's generator, pointed at http://localhost:8001/api/v1/transactions
docker compose exec redis redis-cli --scan --pattern 'vel:acct:*:1h' | head -3
```

If M1's generator writes straight to the database instead of calling the API, point it at `POST /api/v1/transactions` now (header `X-Service-Key`); otherwise Redis never sees the stream and the features stay empty.

```
$ git add -A && git commit -m "[M2-AegisFlow] feat: transaction ingestion with edge upserts and online features"
```

2.8 API contract tests

tests/m2/api/test_m2_api.py

```
"""HTTP contract with in-memory store + fakeredis: created vs duplicate, retry after a
Redis outage completes features exactly once, validation and auth."""
import uuid

import fakeredis
import pytest
import redis as redis_lib
from fastapi.testclient import TestClient

from ingestion import config, deps
from ingestion.main import app
from ingestion.persistence import MemoryStore

KEY = {"X-Service-Key": config.SERVICE_KEY}
ACCT = str(uuid.uuid4())


def body(**over):
    b = {"id": str(uuid.uuid4()), "ts": "2026-09-01T12:00:00Z", "account_id": ACCT,
         "device_id": str(uuid.uuid4()), "amount": "125.50", "currency": "INR",
         "channel": "card"}
    b.update(over)
    return b


@pytest.fixture()
def wired(monkeypatch):
    store, r = MemoryStore(known_accounts={ACCT}), fakeredis.FakeRedis()
    monkeypatch.setattr(deps, "_store", store)
    monkeypatch.setattr(deps, "_redis", r)
    with TestClient(app) as c:
        yield c, store, r


def test_created_then_duplicate(wired):
    c, store, r = wired
    b = body()
    first = c.post("/api/v1/transactions", json=b, headers=KEY)
    assert first.status_code == 201
    assert first.json() == {"id": b["id"], "decision": None, "duplicate": False}
    again = c.post("/api/v1/transactions", json=b, headers=KEY)
    assert again.status_code == 200 and again.json()["duplicate"] is True
    assert r.zcard(f"vel:acct:{ACCT}:1h") == 1           # counted once
    assert sum(store.edges.values()) == 1                # edge weight not inflated


def test_retry_after_redis_outage_completes_once(wired, monkeypatch):
    c, store, r = wired

    class Down:
        def register_script(self, _):
            def boom(**_):
                raise redis_lib.ConnectionError("down")
            return boom
    b = body()
    monkeypatch.setattr(deps, "_redis", Down())
    assert c.post("/api/v1/transactions", json=b, headers=KEY).status_code == 503
    monkeypatch.setattr(deps, "_redis", r)
    retry = c.post("/api/v1/transactions", json=b, headers=KEY)
    assert retry.status_code == 200 and retry.json()["duplicate"] is True
    assert r.zcard(f"vel:acct:{ACCT}:1h") == 1           # features now present, once


@pytest.mark.parametrize("bad", [{"amount": "10.123"}, {"amount": "-5"},
                                 {"currency": "inr"}, {"channel": "cash"},
                                 {"ts": "2026-09-01T12:00:00"}])
def test_validation(wired, bad):
    c, *_ = wired
    assert c.post("/api/v1/transactions", json=body(**bad), headers=KEY).status_code == 422


def test_unknown_account_is_422(wired):
    c, *_ = wired
    r = c.post("/api/v1/transactions", json=body(account_id=str(uuid.uuid4())), headers=KEY)
    assert r.status_code == 422 and "unknown entity" in r.json()["detail"]


def test_service_key_and_probes(wired):
    c, *_ = wired
    assert c.post("/api/v1/transactions", json=body()).status_code == 401
    assert c.get("/health").json() == {"status": "ok"}
```

tests/m2/api/test_m2_api.py

```
    assert c.get("/ready").json()["status"] == "ready"
    assert b"af_transactions_total" in c.get("/metrics").content
```

terminal

```
PYTHONPATH=services/ingestion/src:ml/src:. pytest -q tests/m2/api
# 9 passed
```

```
$ git add -A && git commit -m "[M2-AegisFlow] test: created vs duplicate, outage retry, validation and auth"
```

2.9 Hourly graph feature job

The edge table becomes an undirected NetworkX graph (a ring shares devices regardless of edge direction). Per account: connected-component size, hop distance to the nearest known-fraud account, and personalised PageRank seeded on known fraud.

ml/src/aegis_ml/features/graph_features.py

```
"""Hourly graph feature job (FR-4). Loads entity_edges into an undirected NetworkX
graph (a ring shares devices regardless of edge direction) and computes, per account:
  component_size  nodes in the account's connected component (accounts+devices+IPs+merchants)
  dist_to_fraud   hop count to the nearest known-fraud account (-1 = unreachable)
  risk_rank       personalised PageRank seeded on known-fraud accounts
Writes gfeat:acct:{id} (TTL 2 h) and one graph_feature_snapshots row per account
(Decision D7), so offline training joins the snapshot that existed at each ts.

    python -m aegis_ml.features.graph_features [--once]"""
import argparse
import os
import time
from datetime import datetime, timezone

import networkx as nx
import pandas as pd

from . import definitions as D

PREFIX = {"account": "acct", "device": "dev", "ip": "ip", "merchant": "mer"}

EDGES_SQL = "SELECT src_type, src_id, dst_type, dst_id, weight FROM entity_edges"
FRAUD_SQL = ("SELECT entity_id FROM known_fraud_entities "
             "WHERE entity_type = 'account' AND confirmed_at <= %s")
SNAP_SQL = ("INSERT INTO graph_feature_snapshots "
            "(entity_id, computed_at, component_size, dist_to_fraud, risk_rank) "
            "VALUES (%s, %s, %s, %s, %s)")


def node(entity_type: str, entity_id) -> str:
    return f"{PREFIX[entity_type]}:{entity_id}"


def build_graph(edges) -> nx.Graph:
    g = nx.Graph()
    for src_type, src_id, dst_type, dst_id, weight in edges:
        a, b = node(src_type, src_id), node(dst_type, dst_id)
        w = float(weight or 1.0)
        if g.has_edge(a, b):                        # same pair via two edge types
            g[a][b]["weight"] += w
        else:
            g.add_edge(a, b, weight=w)
    return g


def compute(g: nx.Graph, fraud_accounts) -> pd.DataFrame:
    accounts = [n for n in g if n.startswith("acct:")]
    size = {}
    for comp in nx.connected_components(g):
        for n in comp:
            size[n] = len(comp)
    sources = [node("account", a) for a in fraud_accounts if node("account", a) in g]
    dist, rank = {}, {}
    if sources:
        # weight=1 per hop: edge weight means link STRENGTH, not distance
        dist = nx.multi_source_dijkstra_path_length(g, sources, weight=lambda u, v, d: 1)
```

ml/src/aegis_ml/features/graph_features.py

```
        rank = nx.pagerank(g, alpha=0.85, personalization={s: 1.0 for s in sources},
                           weight="weight")
    return pd.DataFrame({
        "entity_id": [a.split(":", 1)[1] for a in accounts],
        "component_size": [int(size[a]) for a in accounts],
        "dist_to_fraud": [int(dist[a]) if a in dist else -1 for a in accounts],
        "risk_rank": [float(rank.get(a, 0.0)) for a in accounts],
    })


def publish(conn, r, feats: pd.DataFrame, computed_at: datetime) -> None:
    with conn.cursor() as cur:
        cur.executemany(SNAP_SQL, [(row.entity_id, computed_at, row.component_size,
                                    row.dist_to_fraud, row.risk_rank)
                                   for row in feats.itertuples(index=False)])
    conn.commit()
    p = r.pipeline(transaction=False)
    for row in feats.itertuples(index=False):
        key = f"gfeat:acct:{row.entity_id}"
        p.hset(key, mapping={"component_size": row.component_size,
                             "dist_to_fraud": row.dist_to_fraud,
                             "risk_rank": repr(row.risk_rank),
                             "computed_at": computed_at.isoformat()})
        p.expire(key, D.GRAPH_TTL_S)
    p.execute()


def run_once(conn, r) -> int:
    now = datetime.now(timezone.utc)
    g = build_graph(conn.execute(EDGES_SQL).fetchall())
    fraud = [row[0] for row in conn.execute(FRAUD_SQL, (now,)).fetchall()]
    feats = compute(g, fraud)
    publish(conn, r, feats, now)
    return len(feats)


def main():
    import psycopg
    import redis
    ap = argparse.ArgumentParser()
    ap.add_argument("--once", action="store_true")
    ap.add_argument("--interval-s", type=int, default=3600)
    a = ap.parse_args()
    dsn = os.environ["DATABASE_URL"].replace("postgresql+psycopg://", "postgresql://", 1)
    r = redis.Redis.from_url(os.getenv("REDIS_URL", "redis://localhost:6379/0"))
    while True:
        started = time.monotonic()
        try:
            with psycopg.connect(dsn) as conn:
                print(f"graph features for {run_once(conn, r)} accounts", flush=True)
        except Exception as exc:  # noqa: BLE001
            print(f"graph pass failed: {exc}", flush=True)
        if a.once:
            break
        time.sleep(max(0, a.interval_s - (time.monotonic() - started)))


if __name__ == "__main__":
    main()
```

tests/m2/unit/test_m2_graph.py

```
"""Graph features on a ring fixture: A, B, C share device D; A is known fraud;
E is alone on device F."""
from aegis_ml.features.graph_features import build_graph, compute

EDGES = [("account", "A", "device", "D", 1), ("account", "B", "device", "D", 3),
         ("account", "C", "device", "D", 1), ("account", "E", "device", "F", 1),
         ("account", "B", "device", "D", 1)]


def test_ring_fixture():
    f = compute(build_graph(EDGES), ["A"]).set_index("entity_id")
    assert f.loc[["A", "B", "C"], "component_size"].tolist() == [4, 4, 4]
    assert f.loc["E", "component_size"] == 2
    assert f.loc[["A", "B", "C", "E"], "dist_to_fraud"].tolist() == [0, 2, 2, -1]
    # E cannot be reached from A, so its rank is ~0 (PageRank converges iteratively)
    assert f.loc["E", "risk_rank"] < 1e-4 < f.loc["C", "risk_rank"] < f.loc["B", "risk_rank"]


def test_hop_count_ignores_edge_weight():
```

tests/m2/unit/test_m2_graph.py

```
    heavy = [("account", "A", "device", "D", 1000), ("account", "B", "device", "D", 1000)]
    f = compute(build_graph(heavy), ["A"]).set_index("entity_id")
    assert f.loc["B", "dist_to_fraud"] == 2


def test_no_known_fraud_gives_defaults():
    f = compute(build_graph(EDGES), [])
    assert (f["dist_to_fraud"] == -1).all() and (f["risk_rank"] == 0.0).all()
```

> **Edge weight is strength, not distance**
> NetworkX's shortest-path functions read the `weight` attribute as distance by default, so a heavily used link would look far away. The job passes `weight=lambda u, v, d: 1` for hop counts and keeps `weight` for PageRank, where strength is the right meaning.

terminal

```
PYTHONPATH=services/ingestion/src:ml/src:. pytest -q tests/m2/unit/test_m2_graph.py
# 3 passed
```

```
$ git add -A && git commit -m "[M2-AegisFlow] feat: hourly graph features with snapshots"
```

2.10 Known-fraud seed and the point-in-time DVC stage

Known fraud comes only from labels dated before cutoff T0 (default: the 10th percentile of transaction times). M3's training split must start at or after T0, so the graph features it trains on never contain the answer.

db/seed/seed_known_fraud.py

```
"""Seed known_fraud_entities from labels dated BEFORE cutoff T0 only (Decision D5).
Simulates "previously confirmed" fraud; graph features may use it, evaluation may not
overlap it. M3 must start its train split at or after T0.

LABEL_SQL assumes M1's ground_truth_labels has (transaction_id, is_fraud). If M1 named
the columns differently, edit LABEL_SQL only.
Usage: DATABASE_URL=... [KNOWN_FRAUD_CUTOFF=2026-09-08T00:00:00Z] python db/seed/seed_known_fraud.py"""
import os
import sys
from datetime import datetime

import psycopg

LABEL_SQL = """
SELECT DISTINCT t.account_id, min(t.ts) OVER (PARTITION BY t.account_id) AS confirmed_at
FROM ground_truth_labels g JOIN transactions t ON t.id = g.transaction_id
WHERE g.is_fraud AND t.ts < %s"""

DEFAULT_CUTOFF_SQL = """
SELECT percentile_disc(0.10) WITHIN GROUP (ORDER BY ts) FROM transactions"""


def main() -> int:
    dsn = os.environ["DATABASE_URL"].replace("postgresql+psycopg://", "postgresql://", 1)
    with psycopg.connect(dsn) as conn:
        env_cut = os.getenv("KNOWN_FRAUD_CUTOFF")
        cutoff = (datetime.fromisoformat(env_cut.replace("Z", "+00:00")) if env_cut
                  else conn.execute(DEFAULT_CUTOFF_SQL).fetchone()[0])
        if cutoff is None:
            print("no transactions yet - run the generator first", file=sys.stderr)
            return 2
        rows = conn.execute(LABEL_SQL, (cutoff,)).fetchall()
        conn.execute("DELETE FROM known_fraud_entities WHERE source = 'seed_pre_T0'")
        with conn.cursor() as cur:
            cur.executemany(
                "INSERT INTO known_fraud_entities (entity_type, entity_id, confirmed_at, source) "
                "VALUES ('account', %s, %s, 'seed_pre_T0') ON CONFLICT DO NOTHING", rows)
        conn.commit()
    print(f"T0 = {cutoff.isoformat()}  ->  {len(rows)} known-fraud accounts seeded")
    print("Record T0 in params.yaml (features.known_fraud_cutoff); M3's train split starts here.")
    return 0


if __name__ == "__main__":
```

db/seed/seed_known_fraud.py

```
    sys.exit(main())
```

dvc.yaml (append under stages:)

```
# Append under `stages:` in dvc.yaml. The stage reads the live tables, so DVC cannot
# see when they change: always_changed forces it to rerun on every `dvc repro`.
  features_pit:
    cmd: PYTHONPATH=ml/src python -m aegis_ml.features.offline --out data/features/features_pit.parquet
    deps:
      - ml/src/aegis_ml/features
    outs:
      - data/features/features_pit.parquet
    always_changed: true
```

terminal

```
make seed-fraud
# T0 = 2026-09-08T...  ->  N known-fraud accounts seeded
make graph-once
# graph features for N accounts
make features-pit
# N rows x 19 features (feature set <version>) -> data/features/features_pit.parquet
```

Record the printed T0 in params.yaml as `features: {known_fraud_cutoff: <T0>}` and commit it - M3 reads it to place the train split.

```
$ git add -A && git commit -m "[M2-AegisFlow] feat: pre-T0 known-fraud seed and point-in-time DVC stage"
```

2.11 Database integration tests

Against the real database: transactions are idempotent, edge weights count distinct transactions only, an unknown account is rejected through the foreign key, and the graph job writes snapshots and Redis hashes. The tests create and delete their own rows.

tests/m2/integration/test_m2_db.py

```
"""Against the real database (make migrate done). Skipped without DATABASE_URL.
Creates its own accounts/devices and removes everything it wrote."""
import os
import uuid
from datetime import datetime, timedelta, timezone
from decimal import Decimal

import pytest

pytestmark = pytest.mark.skipif(not os.getenv("DATABASE_URL"), reason="needs the stack")

import psycopg  # noqa: E402
import redis  # noqa: E402
from sqlalchemy import create_engine  # noqa: E402

from aegis_ml.features import graph_features as G  # noqa: E402
from ingestion.persistence import PgStore, UnknownEntity  # noqa: E402
from ingestion.schemas import TransactionIn  # noqa: E402


@pytest.fixture()
def conn():
    dsn = os.environ["DATABASE_URL"].replace("postgresql+psycopg://", "postgresql://", 1)
    with psycopg.connect(dsn) as c:
        yield c


@pytest.fixture()
def ring(conn):
    """Accounts A, B, C on one device; A is known fraud. Cleaned up afterwards."""
    accts = [uuid.uuid4() for _ in range(3)]
    dev = uuid.uuid4()
    for a in accts:
        conn.execute("INSERT INTO accounts (id) VALUES (%s)", (a,))
    conn.execute("INSERT INTO devices (id) VALUES (%s)", (dev,))
    conn.execute("INSERT INTO known_fraud_entities VALUES ('account', %s, now() - interval "
                 "'1 day', 'it_test')", (accts[0],))
    conn.commit()
```

tests/m2/integration/test_m2_db.py

```
    yield accts, dev
    ids = [str(a) for a in accts]
    conn.execute("DELETE FROM graph_feature_snapshots WHERE entity_id = ANY(%s::uuid[])", (ids,))
    conn.execute("DELETE FROM entity_edges WHERE src_id = ANY(%s::uuid[])", (ids,))
    conn.execute("DELETE FROM transactions WHERE account_id = ANY(%s::uuid[])", (ids,))
    conn.execute("DELETE FROM known_fraud_entities WHERE source = 'it_test'")
    conn.execute("DELETE FROM accounts WHERE id = ANY(%s::uuid[])", (ids,))
    conn.execute("DELETE FROM devices WHERE id = %s", (dev,))
    conn.commit()


def txn(account, device, minute):
    return TransactionIn(id=uuid.uuid4(), account_id=account, device_id=device,
                         ts=datetime(2026, 9, 1, 12, minute, tzinfo=timezone.utc),
                         amount=Decimal("10.00"), currency="INR", channel="card")


def test_store_is_idempotent_and_upserts_edges(conn, ring):
    (a, b, c), dev = ring
    store = PgStore(create_engine(os.environ["DATABASE_URL"]))
    t1 = txn(a, dev, 0)
    assert store.store(t1) is True and store.store(t1) is False
    assert store.store(txn(a, dev, 1)) is True
    w = conn.execute("SELECT weight FROM entity_edges WHERE src_id = %s AND dst_id = %s",
                     (a, dev)).fetchone()[0]
    assert w == 2                       # two distinct transactions, retry not counted
    with pytest.raises(UnknownEntity):
        store.store(txn(uuid.uuid4(), dev, 2))


def test_graph_job_publishes_snapshots_and_redis(conn, ring):
    (a, b, c), dev = ring
    store = PgStore(create_engine(os.environ["DATABASE_URL"]))
    for i, acct in enumerate((a, b, c)):
        store.store(txn(acct, dev, 10 + i))
    r = redis.Redis.from_url(os.getenv("REDIS_URL", "redis://localhost:6379/0"))
    G.run_once(conn, r)
    row = conn.execute("SELECT component_size, dist_to_fraud FROM graph_feature_snapshots "
                       "WHERE entity_id = %s ORDER BY computed_at DESC LIMIT 1", (b,)).fetchone()
    assert row[0] >= 4 and row[1] == 2
    assert r.hget(f"gfeat:acct:{b}", "dist_to_fraud") == b"2"
    assert 0 < r.ttl(f"gfeat:acct:{b}") <= 7200
    r.delete(*[f"gfeat:acct:{x}" for x in (a, b, c)])


def test_unique_edge_index_exists(conn):
    assert conn.execute("SELECT 1 FROM pg_indexes WHERE indexname = 'uq_entity_edges_key'"
                        ).fetchone() is not None
```

terminal

```
make test-m2-int
# 3 passed
```

```
$ git add -A && git commit -m "[M2-AegisFlow] test: database integration for edges, FK and graph job"
```

2.12 Containers, verification and the v0.2.0-M2 tag

The ingestion image carries `ml/src/aegis_ml` too, because the Lua path is compiled from `definitions.py` - one source, even in production.

services/ingestion/requirements.txt

```
fastapi==0.115.6
uvicorn[standard]==0.32.1
pydantic==2.9.2
sqlalchemy==2.0.36
psycopg[binary]==3.2.3
redis==5.2.0
loguru==0.7.3
prometheus-client==0.21.1
```

services/ingestion/Dockerfile

```
# services/ingestion/Dockerfile - build from the repo root. The image carries
# ml/src/aegis_ml too, because online_features.py compiles definitions.py (one source).
FROM python:3.12-slim
WORKDIR /app
COPY services/ingestion/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY services/ingestion/src /app/services/ingestion/src
COPY ml/src /app/ml/src
ENV PYTHONPATH=/app/services/ingestion/src:/app/ml/src
EXPOSE 8001
HEALTHCHECK --interval=15s --retries=10 CMD python -c \
  "import urllib.request as u; u.urlopen('http://localhost:8001/health')"
CMD ["uvicorn", "ingestion.main:app", "--host", "0.0.0.0", "--port", "8001"]
```

ml/requirements-graph.txt

```
networkx==3.4.2
scipy==1.14.1
numpy==1.26.4
pandas==2.2.3
pyarrow==18.1.0
sqlalchemy==2.0.36
psycopg[binary]==3.2.3
redis==5.2.0
```

ml/Dockerfile.graph

```
# ml/Dockerfile.graph - hourly graph feature job. Build from the repo root.
FROM python:3.12-slim
WORKDIR /app
COPY ml/requirements-graph.txt .
RUN pip install --no-cache-dir -r requirements-graph.txt
COPY ml/src /app/ml/src
ENV PYTHONPATH=/app/ml/src
CMD ["python", "-m", "aegis_ml.features.graph_features"]
```

docker-compose.m2.yml

```
# docker-compose.m2.yml - layered on the M1 stack:
#   docker compose -f docker-compose.yml -f docker-compose.m2.yml up -d --build
# Assumes M1 services named `postgres` and `redis`; rename below if yours differ.
services:
  ingestion:
    build:
      context: .
      dockerfile: services/ingestion/Dockerfile
    container_name: aegisflow-ingestion
    depends_on: [postgres, redis]
    environment:
      DATABASE_URL: postgresql+psycopg://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}
      REDIS_URL: redis://redis:6379/0
      SERVICE_KEY: ${SERVICE_KEY:?set SERVICE_KEY in .env}
    ports: ["8001:8001"]
    restart: unless-stopped

  graph-job:
    build:
      context: .
      dockerfile: ml/Dockerfile.graph
    container_name: aegisflow-graph-job
    depends_on: [postgres, redis]
    environment:
      DATABASE_URL: postgresql+psycopg://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}
      REDIS_URL: redis://redis:6379/0
    restart: unless-stopped
```

docs/adr/0001-feature-store.md

```
# ADR 0001 - Feature store parity rules (M2)

Status: accepted (M2)

- D1 Event time: windows use the transaction's ts, passed into Lua; never Redis TIME.
  The generator replays at 10x speed, so wall-clock windows would disagree with offline.
- D2 Velocity windows are (ts - w, ts], current transaction included; ties are ordered by
  (ts, id) in both paths.
- D3 Idempotency: SET seen:txn:{id} NX inside the script; amounts in integer minor units.
- D4 Welford profile in feat:acct:{id}; amount_zscore uses the profile BEFORE the amount.
- D5 Known fraud only from labels dated before cutoff T0 (known_fraud_entities); never
  ground_truth_labels at serve time.
- D6 Unique index on entity_edges (src_type, src_id, dst_type, dst_id, edge_type); edges
  are upserted only when the transaction row is new.
- D7 Hourly graph_feature_snapshots; offline joins the latest snapshot with
  computed_at <= ts.
- D8 Entity upsert endpoints are deferred; the M1 generator keeps seeding entities.
  M1's entity columns are not fixed by the spec, so a generic upsert could violate them.

Floats cross Redis as %.17g strings so they round-trip bit-exactly; the parity test
demands exact equality.
```

scripts/verify_m2.py

```
"""M2 definition-of-done checks. Exit 0 means ready to tag v0.2.0-M2.
Needs the stack up (make up-m2), migrations applied, DATABASE_URL/REDIS_URL exported."""
import os
import pathlib
import subprocess
import sys
import urllib.request

ROOT = pathlib.Path(__file__).resolve().parents[1]
FILES = [
    "ml/src/aegis_ml/features/definitions.py", "ml/src/aegis_ml/features/offline.py",
    "ml/src/aegis_ml/features/graph_features.py",
    "services/ingestion/src/ingestion/config.py", "services/ingestion/src/ingestion/schemas.py",
    "services/ingestion/src/ingestion/persistence.py",
    "services/ingestion/src/ingestion/online_features.py",
    "services/ingestion/src/ingestion/deps.py", "services/ingestion/src/ingestion/main.py",
    "services/ingestion/src/ingestion/api/routes.py", "services/ingestion/Dockerfile",
    "services/ingestion/requirements.txt", "ml/Dockerfile.graph", "ml/requirements-graph.txt",
    "db/migrations/versions/m2_0001_feature_store.py", "db/seed/seed_known_fraud.py",
    "docker-compose.m2.yml", "docs/adr/0001-feature-store.md",
    "tests/m2/unit/test_m2_parity.py", "tests/m2/unit/test_m2_offline.py",
    "tests/m2/unit/test_m2_graph.py", "tests/m2/api/test_m2_api.py", "tests/m2/integration/test_m2_db.py",
]
results = []


def ok(name, cond, detail=""):
    results.append(bool(cond))
    print(f"[{'OK ' if cond else 'XX '}] {name} {detail}")


for f in FILES:
    ok(f"file {f}", (ROOT / f).exists())
env = {**os.environ, "PYTHONPATH": "services/ingestion/src:ml/src:."}
r = subprocess.run([sys.executable, "-m", "pytest", "-q", "tests/m2/unit", "tests/m2/api"],
                   cwd=ROOT, env=env)
ok("unit + API tests (incl. exact parity over 5,000 transactions)", r.returncode == 0)

if not os.getenv("DATABASE_URL"):
    ok("DATABASE_URL exported", False, "(see env.m2.example)")
else:
    r = subprocess.run([sys.executable, "-m", "pytest", "-q", "tests/m2/integration"],
                       cwd=ROOT, env=env)
    ok("integration tests (edges, FK, graph job)", r.returncode == 0)
    import psycopg
    import redis
    dsn = os.environ["DATABASE_URL"].replace("postgresql+psycopg://", "postgresql://", 1)
    with psycopg.connect(dsn) as c:
        n_tx = c.execute("SELECT count(*) FROM transactions").fetchone()[0]
        ok("transactions stored", n_tx > 0, f"({n_tx})")
        dup = c.execute("SELECT count(*) FROM (SELECT 1 FROM entity_edges GROUP BY src_type, "
                        "src_id, dst_type, dst_id, edge_type HAVING count(*) > 1) d").fetchone()[0]
        ok("no duplicate edges", dup == 0)
        snap = c.execute("SELECT max(computed_at) > now() - interval '65 minutes' "
                         "FROM graph_feature_snapshots").fetchone()[0]
        ok("graph snapshot within the last hour", bool(snap))
```

scripts/verify_m2.py

```
        kf = c.execute("SELECT count(*) FROM known_fraud_entities").fetchone()[0]
        ok("known fraud seeded (pre-T0)", kf > 0, f"({kf})")
    rds = redis.Redis.from_url(os.getenv("REDIS_URL", "redis://localhost:6379/0"))
    ok("feat:acct:* keys present", next(rds.scan_iter("feat:acct:*", count=1000), None))
    ok("gfeat:acct:* keys present", next(rds.scan_iter("gfeat:acct:*", count=1000), None))

base = os.getenv("INGEST_URL", "http://localhost:8001")
for path in ("/health", "/ready"):
    try:
        code = urllib.request.urlopen(base + path, timeout=5).status
    except Exception as exc:  # noqa: BLE001
        code = str(exc)[:60]
    ok(f"ingestion {path}", code == 200, f"({code})")

print(f"\n{sum(results)}/{len(results)} checks passed")
sys.exit(0 if all(results) else 1)
```

terminal

```
make up-m2
make generate                       # stream into the containerised API for a minute
make seed-fraud && make graph-once
make m2                             # unit + API -> integration -> verify
# ...
# 33/33 checks passed
git add -A
git commit -m "[M2-AegisFlow] chore: containers and M2 verification script"
git checkout develop && git merge --no-ff feature/m2-feature-store
git checkout main && git merge --no-ff develop
git tag -a v0.2.0-M2 -m "AegisFlow Milestone 2 - feature store and entity graph"
git push origin main develop --tags
```

M2 definition of done

- `make m2` runs green and `verify_m2.py` exits 0.
- `definitions.py` is the only place features are declared; both paths import it.
- Exact parity over 5,000 replayed transactions on fakeredis and on a real Redis.
- A repeated transaction id changes nothing in PostgreSQL or Redis.
- The graph job runs hourly; `gfeat:acct:*` exists and snapshots gain a run each hour.
- Nothing on the serving path reads `ground_truth_labels`; known fraud is pre-T0 only.
- ADR 0001 (D1-D8) committed; tag v0.2.0-M2 pushed.

Commit log for M2

git log --oneline (expected)

```
[M2-AegisFlow] config: dependencies, settings and M2 make targets
[M2-AegisFlow] feat: unique edge key, graph snapshots and known-fraud tables
[M2-AegisFlow] feat: single-source feature definitions
[M2-AegisFlow] feat: offline point-in-time builder with hand-computed tests
[M2-AegisFlow] feat: atomic idempotent online feature script
[M2-AegisFlow] test: exact offline/online parity over a replayed stream
[M2-AegisFlow] feat: transaction ingestion with edge upserts and online features
[M2-AegisFlow] test: created vs duplicate, outage retry, validation and auth
[M2-AegisFlow] feat: hourly graph features with snapshots
[M2-AegisFlow] feat: pre-T0 known-fraud seed and point-in-time DVC stage
[M2-AegisFlow] test: database integration for edges, FK and graph job
[M2-AegisFlow] chore: containers and M2 verification script
```

What M3 picks up from here

M3 trains on `data/features/features_pit.parquet` with time-based splits that start at or after T0 (weeks 1-6 train, 7-8 test per spec section 14) and stamps `FEATURE_SET_VERSION` into each model card. IEEE-CIS keeps its own M1-prepared features; this store covers the synthetic and live transaction schema. M4's scoring service reads the same Redis keys in one pipelined round trip and fills the `decision` field this API leaves null.